

CS-GY 6923: Lecture 9

Neural Nets

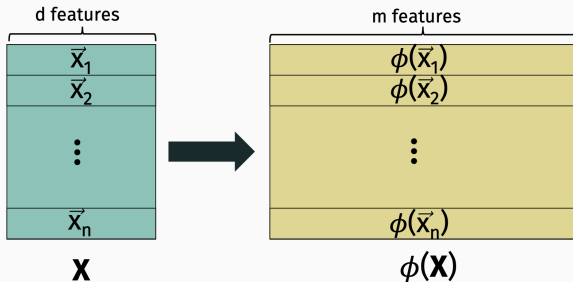
NYU Tandon School of Engineering, Akbar Rafiey

Non-linear methods

- Previous methods studied (regression, logistic regression) are considered linear methods.
- They make predictions based on $\langle \mathbf{x}, \beta \rangle$ – i.e. based on weighted sums of features.
- Next part of the course: we move on to non-linear methods. Specifically, **kernel methods** and **neural networks**.
- Both are very closely related to feature transformations, which was one technique we saw for using linear methods to learn non-linear concepts.

Feature transformation

Let $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^m$ be a feature transformation map that takes in a data point $\mathbf{x}$ and returns its transformed version $\phi(\mathbf{x})$.



Challenges in feature transformation

There are two major issues with this:

- Coming up with an explicit and simple feature transformation map $\phi(\mathbf{x})$ is not that simple (sometimes we can do it ad-hoc and specific to a problem)
- Sometimes the so-called hidden dimension m is very large and computationally prohibitive to use ϕ .

We will use a reparameterization trick to overcome these challenges.

Reparameterization trick

For simplicity, let's just consider the binary cross entropy/logistic regression loss:

$$-\sum_{j=1}^n y_j \log(h(\mathbf{X}\beta)_j) + (1 - y_j) \log(1 - h(\mathbf{X}\beta)_j)$$

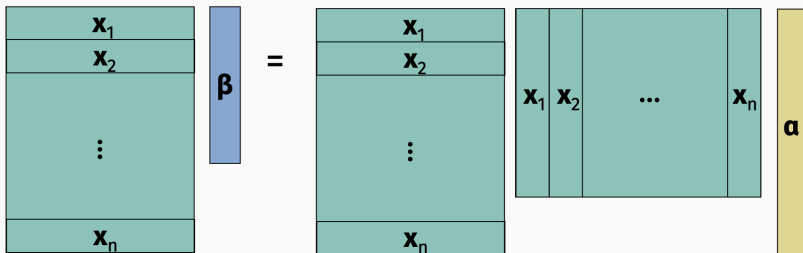
where $h(z) = \frac{1}{1+e^{-z}}$.

Reparameterization trick

Reminder from linear algebra: Without loss of generality, can assume that β lies in the row span of $\mathbf{X}$.

So for any $\beta \in \mathbb{R}^d$, there exists a vector $\alpha \in \mathbb{R}^n$ such that:

$$\mathbf{X}\beta = \mathbf{X}\mathbf{X}^T\alpha.$$



Reparameterization trick

Logistic Regression Equivalent Formulation: Given data matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ and binary label vector $\mathbf{y} \in \{0, 1\}^n$ for class i , find $\alpha \in \mathbb{R}^n$ to minimize the loss:

$$-\sum_{j=1}^n y_j \log(h(\mathbf{X}\mathbf{X}^T \alpha)_j) + (1 - y_j) \log(1 - h(\mathbf{X}\mathbf{X}^T \alpha)_j)$$

Can still be minimized via gradient descent:

$$\nabla L(\alpha) = \mathbf{X}\mathbf{X}^T (h(\mathbf{X}\mathbf{X}^T \alpha) - \mathbf{y}).$$

Reparameterization trick

If we use a non-linear data transformation ϕ , then the loss is:

$$-\sum_{j=1}^n y_j \log(h(\phi(\mathbf{X})\phi(\mathbf{X})^T \boldsymbol{\alpha})_j) + (1 - y_j) \log(1 - h(\phi(\mathbf{X})\phi(\mathbf{X})^T \boldsymbol{\alpha})_j)$$

Kernel matrix

$\mathbf{K} = \phi(\mathbf{X})\phi(\mathbf{X})^T$ is called the kernel Gram matrix.

$$\phi(\mathbf{X}) \phi(\mathbf{X})^T = \begin{array}{|c|} \hline \phi(\vec{x}_1) \\ \hline \phi(\vec{x}_2) \\ \hline \vdots \\ \hline \phi(\vec{x}_n) \\ \hline \end{array} \begin{array}{|c|c|c|c|} \hline \phi(\vec{x}_1) & \phi(\vec{x}_2) & \dots & \phi(\vec{x}_n) \\ \hline \end{array} = \begin{array}{|c|} \hline k(\vec{x}_i, \vec{x}_j) \\ \hline \end{array}$$

$\mathbf{K}$

Kernel functions from feature transformation

A positive semidefinite (PSD) kernel is any similarity function with the following form:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

where $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^m$ is a some feature transformation function.

Kernel functions and feature transformation

Not all similarity metrics are positive semidefinite (PSD), but all of the ones we saw earlier are:

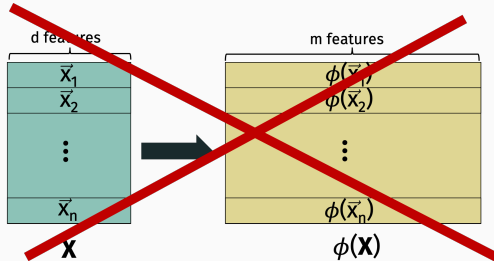
- Gaussian RBF Kernel: $k(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x}-\mathbf{y}\|_2^2/\sigma^2}$
- Laplace Kernel: $k(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x}-\mathbf{y}\|_2/\sigma}$
- Polynomial Kernel: $k(\mathbf{x}, \mathbf{y}) = (\langle \mathbf{x}, \mathbf{y} \rangle + 1)^q$.

And there are many more...

Kernel trick

We never need to actually compute $\phi(\mathbf{x}_1), \dots, \phi(\mathbf{x}_n)$ explicitly!

- For training we just need the kernel matrix $\mathbf{K}$, which requires computing $k(\mathbf{x}_i, \mathbf{x}_j)$ for all i, j .



We can always work with a finite sized $n \times n$ matrix.

Take away:

- Logistic regression can be combined with any positive semidefinite kernel matrix, and the model can be trained in time independent of the transform dimension m .

Kernel trick: prediction

Prediction:

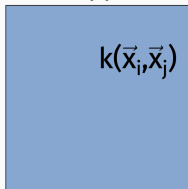
- Prediction can also be done efficiently. For a new input $\mathbf{x}_{new}$, we need to compute:

$$\begin{aligned}\langle \phi(\mathbf{x}_{new}), \boldsymbol{\beta} \rangle &= \langle \phi(\mathbf{x}_{new}), \phi(\mathbf{X})^T \boldsymbol{\alpha} \rangle \\ &= \langle \phi(\mathbf{x}_{new}), \sum_{i=1}^n \phi(\mathbf{x}_i) \alpha_i \rangle = \sum_{i=1}^n \alpha_i \langle \phi(\mathbf{x}_{new}), \phi(\mathbf{x}_i) \rangle.\end{aligned}$$

Each term in the sum $\langle \phi(\mathbf{x}_{new}), \phi(\mathbf{x}_i) \rangle = k(\mathbf{x}_{new}, \mathbf{x}_i)$ can be computed without explicit feature transformation.

Beyond the kernel trick

The kernel matrix $\mathbf{K}$ is still $n \times n$ though which is huge when the size of the training set n is large. Has made the kernel trick less appealing in some modern ML applications.



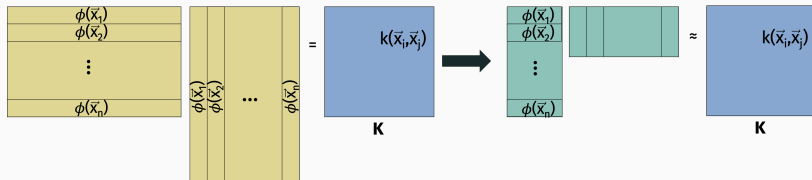
$\mathbf{K}$

There is an inherent quadratic dependence on n in the computational and space complexity of kernel methods.

- 10,000 data points $\rightarrow$ runtime scales as $\sim 100,000,000$, $\mathbf{K}$ takes 800MB of space.
- 1,000,000 data points $\rightarrow$ runtime scales as $\sim 10^{12}$, $\mathbf{K}$ takes 8TB of space.

Beyond the kernel trick

Many algorithmic advances in recent years partially address this computational challenge (random Fourier features methods, Nystrom methods, etc.)



Kernel regression

The kernel trick can also be applied outside of classification. E.g. to regression:

$$\min_{\beta} \|\mathbf{X}\beta - \mathbf{y}\|_2^2 + \lambda \|\beta\|_2^2 \rightarrow \min_{\alpha} \|\mathbf{X}\mathbf{X}^T \alpha - \mathbf{y}\|_2^2 + \lambda \|\mathbf{X}^T \alpha\|_2^2$$

Replace $\mathbf{X}\mathbf{X}^T$ by kernel matrix $\mathbf{K}$ during training.

Prediction:

$$y_{new} = \sum_{i=1}^n \alpha_i \cdot k(\mathbf{x}_{new}, \mathbf{x}_i).$$

Added benefit: Relatively numerically stable. E.g. is a much better option for performing multivariate or even single variate polynomial regression than direct feature expansion.

Neural Nets

Key Concept

Things we have learned so far:

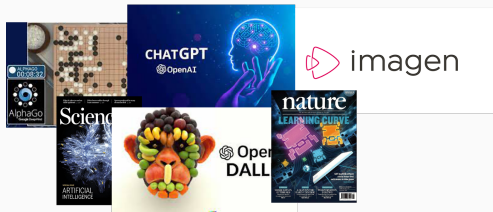
- Choose good features or a good kernel.
- Use optimization to find best model given those features.

Neural network approach:

- Learn good features and a good model simultaneously.

Neural networks

The hot-topic in machine learning right now. With no sign of slowing down.

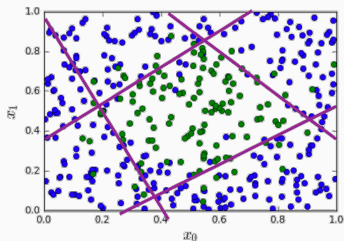
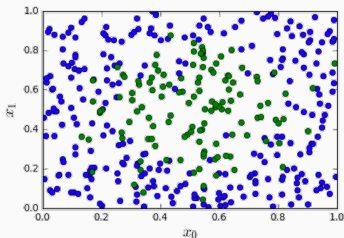


Focus of investment at universities, government research labs, funding agencies, and large tech companies.

Studied since the 1940s/50s. **Why the recent attention?** More on history of neural networks shortly.

Simple motivating example

Classification when data is not linearly separable:

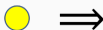
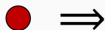
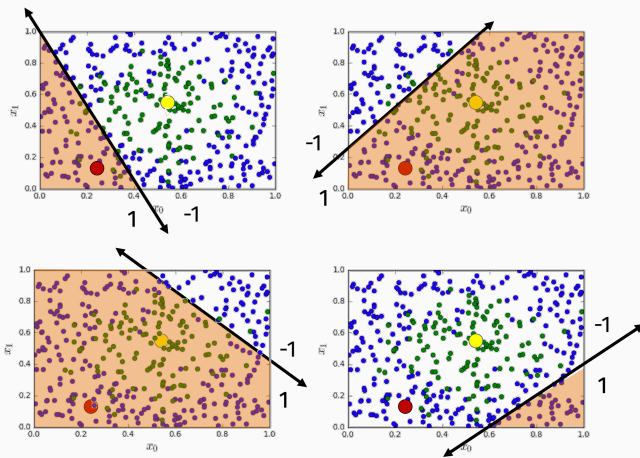


Could use feature transformations or a non-linear kernel.

Alternative approach: Divide the space up into regions using multiple linear classifiers.

Simple motivating example

For each linear classifier β , add a new $-1, 1$ feature for every example $\mathbf{x} = [x_0, x_1]$ depending on the sign of $\langle \mathbf{x}, \beta \rangle$.



Simple motivating example

$$\begin{bmatrix} .2, .8, \\ .5, .5 \\ \vdots \\ .5, 1 \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_n \end{bmatrix} \Rightarrow \begin{bmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \\ \vdots \\ \mathbf{u}_n \end{bmatrix} = \begin{bmatrix} -1, -1, +1, -1 \\ -1, +1, +1, -1 \\ \vdots \\ -1, -1, -1, -1 \end{bmatrix}$$

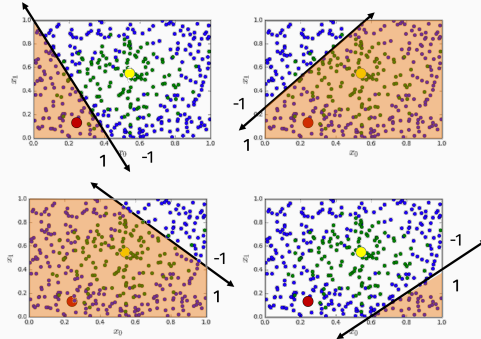
Question: After data transformation, how should we map each new vector $\mathbf{u}_i$ to a class label?

$$\begin{bmatrix} -1, -1, +1, -1 \\ -1, +1, +1, -1 \\ \vdots \\ -1, -1, -1, -1 \end{bmatrix} \xrightarrow{?} \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

Simple motivating example

Our machine learning algorithms needs to **learn two things**:

- The original linear functions which divide our data set into regions (their slopes + intercepts).



- Another linear function which maps our new features to an output class probability.

Possible model

Input: $\mathbf{x} = x_1, \dots, x_{N_I}$

Model: $f(\mathbf{x}, \Theta)$:

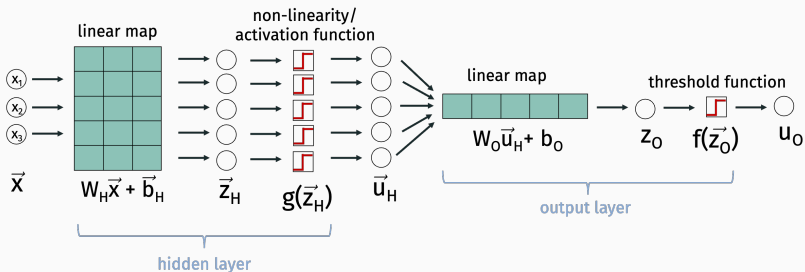
- $\mathbf{z}_H \in \mathbb{R}^{N_H} = \mathbf{W}_H \mathbf{x} + \mathbf{b}_H$.
- $\mathbf{u}_H = \text{sign}(\mathbf{z}_H)$
- $z_O \in \mathbb{R} = \mathbf{W}_O \mathbf{u}_H + \mathbf{b}_O$
- $u_O = \mathbb{1}[z_O]$

Parameters:

- $\Theta = [\mathbf{W}_H \in \mathbb{R}^{N_H \times N_I}, \mathbf{b}_H \in \mathbb{R}^{N_H}, \mathbf{W}_O \in \mathbb{R}^{1 \times N_H}, \mathbf{b}_O \in \mathbb{R}]$.
- $\mathbf{W}_H, \mathbf{W}_O$ are weight matrices and $\mathbf{b}_H, \mathbf{b}_O$ are bias terms that account for the intercepts of our linear functions.

Possible model

Our model is function f which maps $\mathbf{x}$ to a class label u_0 .¹



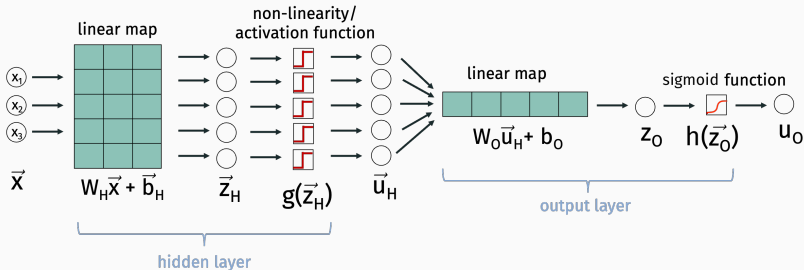
This is called a “multilayer perceptron (MLP)”: one of the oldest types of neural nets. Dates back to Frank Rosenblatt from 1958

- Number of input variables $N_I =$
- Number of hidden variables $N_H =$
- Number of output variables $N_O =$

¹For regression, would cut off at z_0 to get continuous output.

Possible model

Our model is function f which maps $\mathbf{x}$ to a class label u_O .

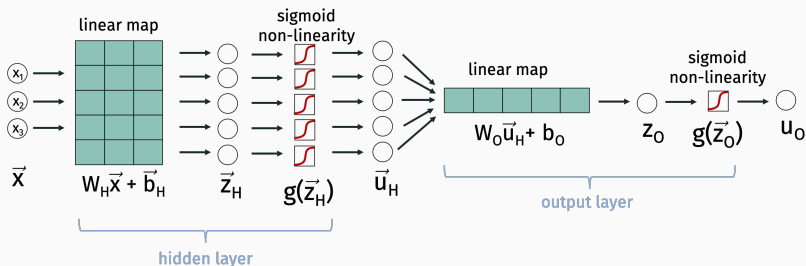


Training the model:

- Choose a loss function $L(f(\mathbf{x}, \Theta), y)$.
- Find optimal parameters: $\Theta^* = \arg \min_{\Theta} \sum_{i=1}^n L(f(\mathbf{x}_i, \Theta), y_i)$ using gradient descent.

Final model

A more typical model uses smoother activation functions, aka non-linearities, which are more amenable to computing gradients. E.g. we might use the **sigmoid function** $g(x) = \frac{1}{1+e^{-x}}$.



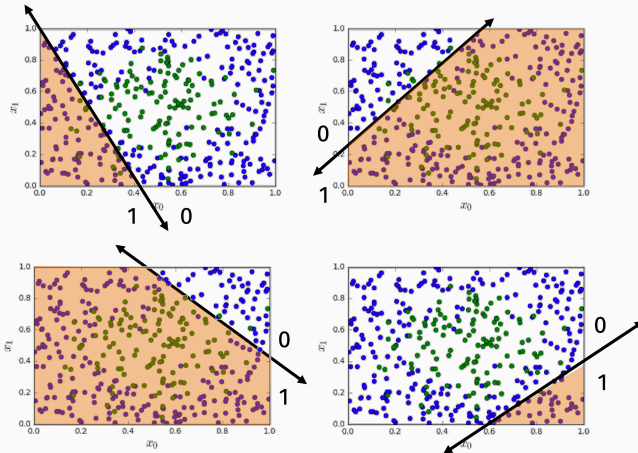
- Use cross-entropy loss:

$$L(f(\mathbf{x}_i, \Theta), y_i) = -y_i \log(f(\mathbf{x}_i, \Theta)) - (1 - y_i) \log(1 - f(\mathbf{x}_i, \Theta))$$

- We will discuss soon how to compute gradients.

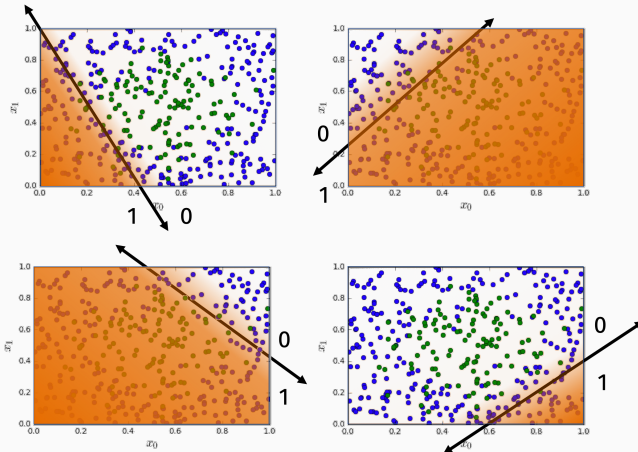
Feature extraction

Features learned using step-function activation are binary, depending on which side of a set of learned hyperplanes each point lies on.



Feature extraction

Features learned using sigmoid activation are real valued in $[0, 1]$.
Mimic binary features.



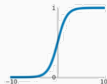
Hyperparameters

Things we can change in this basic classification network:

- More or less hidden variables.
- We could add more layers.
- Different non-linearity/activation function.
- Different loss function.

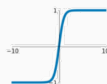
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



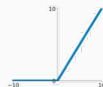
tanh

$$\tanh(x)$$



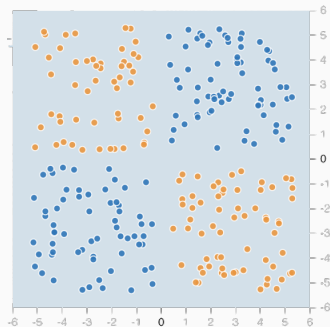
ReLU

$$\max(0, x)$$



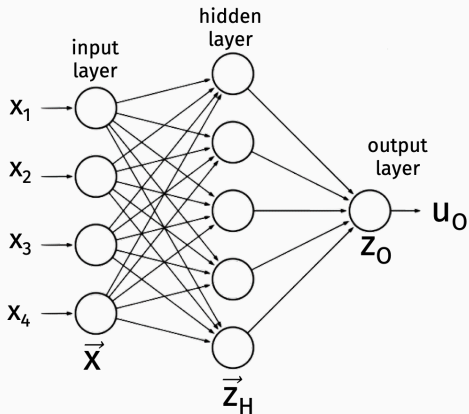
Test your intuition

How many hidden variables (e.g. splitting hyperplanes) would be needed to classify this dataset correctly?

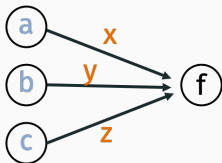


<https://playground.tensorflow.org/>

Another common diagram for a 2-layered network:



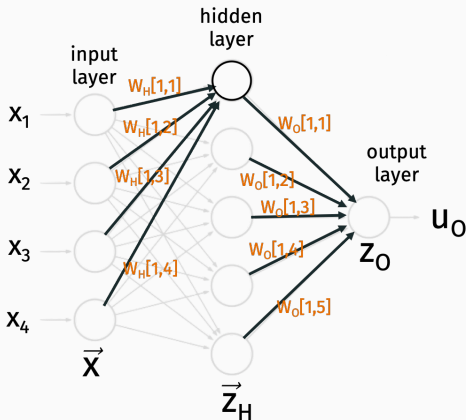
Neural network math:



$$f = ax + by + cz$$

Notation

How to interpret:

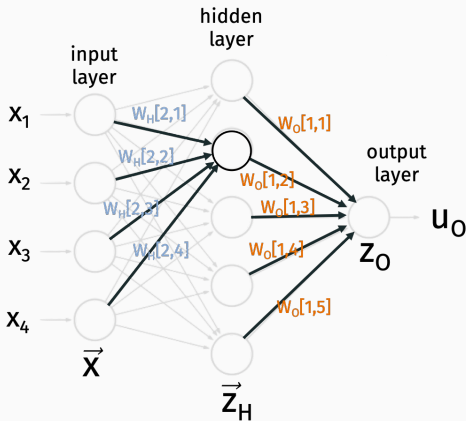


$\mathbf{W}_H$ and $\mathbf{W}_O$ are our weight matrices from before.

Note: This diagram does not explicitly show the bias terms or the non-linear activation functions.

Notation

How to interpret:

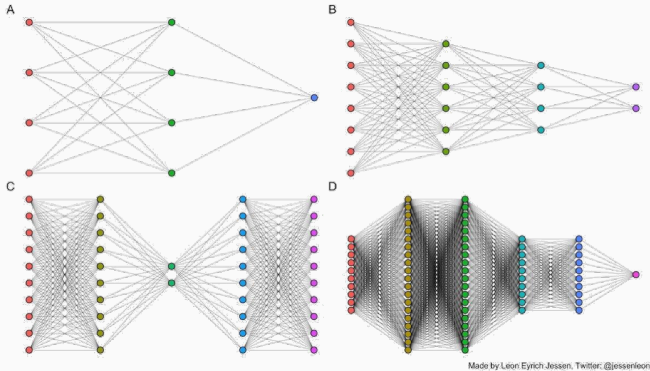


$\mathbf{W}_H$ and $\mathbf{W}_O$ are our weight matrices from before.

Note: This diagram depicts a network with “**fully-connected**” layers. Every variable in layer i is connected to every variable in layer $i + 1$.

Architecture visualization

Effective way of visualize “architecture” of a neural network:



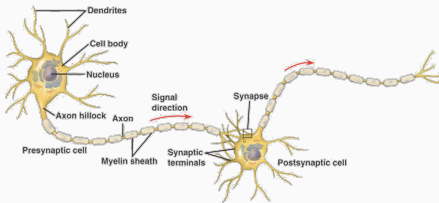
Visualize number of variables, types of connections, number of layers and their relative sizes.

These are all **feedforward** neural networks. No backwards (**recurrent**) connections.

Some history and motivation

Connection to biology

Simplified model of the brain:

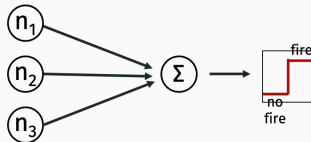


Dendrites: Input electrical current from other neurons.

Axon: Output electrical current to other neurons.

Synapse: Where these two connect.

A neuron “fires” (outputs non-zero electric charge) if it receives enough cumulative electrical input from all neurons connected to it.



Output charge can be positive or negative (excitatory vs. inhibitory).

Inspired early work on neural networks:

- 1940s Donald Hebb proposed a Hebbian learning rule for how brains neurons change over time to allow learning.
- 1950s Frank Rosenblatt's Perceptron is one of the first "artificial" neural networks.
- Continued work throughout the 1960s.

Main issue with neural network methods: They are hard to train. Generally require a lot of computation power. Also pretty finicky: user needs to be careful with initialization, regularization, etc. when training. We have gotten a lot better at resolving these issues though!

Early neural network explosion

Around 1985 several groups (re)-discovered the **backpropagation algorithm** which allows for efficient training of neural nets via **(stochastic) gradient descent**. Along with increased computational power this led to a resurgence of interest in neural network models.

Backpropagation Applied to Handwritten Zip Code Recognition

Y. LeCun
B. Boser
J. S. Denker
D. Henderson
R. E. Howard
W. Hubbard
L. D. Jackel

AT&T Bell Laboratories Holmdel, NJ 07733 USA

The ability of learning networks to generalize can be greatly enhanced by providing constraints from the task domain. This paper demonstrates how such constraints can be integrated into a backpropagation network through the architecture of the network. This approach has been successfully applied to the recognition of handwritten zip code digits provided by the U.S. Postal Service. A single network learns the entire recognition operation, going from the normalized image of the character to the final classification.

Very good performance on problems like digit recognition.

Neural network decline

From 1990s - 2010, kernel methods, SVMs, and probabilistic methods began to dominate the literature in machine learning:

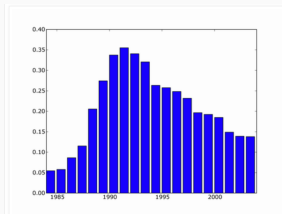
- Work well “out of the box”.
- Relatively easy to understand theoretically.
- Not too computationally expensive for moderately sized datasets.

Fun blog post to check out from 2005:

<http://yaroslavvb.blogspot.com/2005/12/trends-in-machine-learning-according.html>

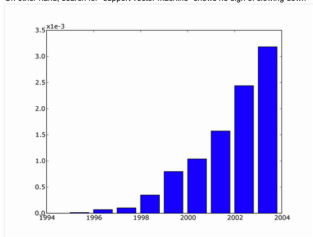
Neural network decline

Finding trends in machine learning by search papers in Google Scholar that match a certain keyword:



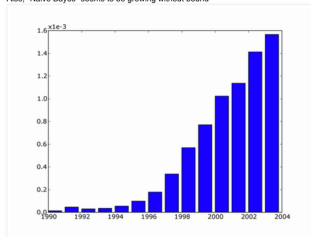
You can see a major upward trend starting around 1985 (that's when Yann LeCun and several others independently rediscovered backpropagation algorithm), peaking in 1992, and going downwards from then.

On other hand, search for "support vector machine" shows no sign of slowing down



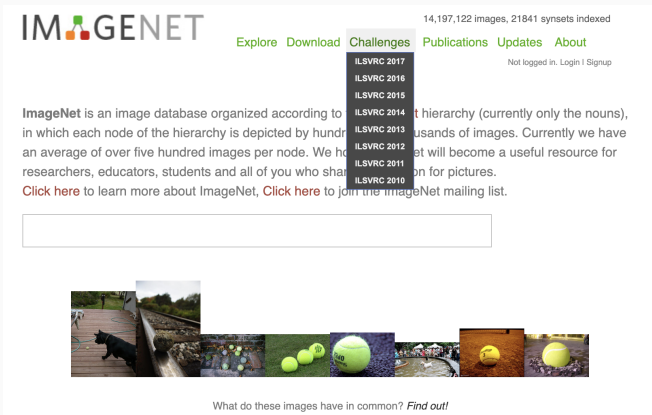
(1995 is when Vapnik and Cortez proposed the algorithm)

Also, "Naive Bayes" seems to be growing without bound



If I were to trust this, I would say that Naive Bayes research the hottest machine learning area right now

All changed with the introduction of AlexNet and the 2012 ImageNet Challenge...



The screenshot shows the ImageNet website homepage. At the top left is the "IMAGENET" logo. To its right are navigation links: "Explore", "Download", "Challenges", "Publications", "Updates", and "About". The "Challenges" link is highlighted with a dark vertical bar containing a list of ILSVRC years from 2010 to 2017. In the top right corner, it says "14,197,122 images, 21841 synsets indexed" and "Not logged in. Login | Signup". The main text describes ImageNet as an image database organized by a hierarchy of synsets, with an average of over five hundred images per node. It mentions that the database will become a useful resource for researchers, educators, students, and anyone who shares a passion for pictures. There are two "Click here" links: one to learn more about ImageNet and another to join the mailing list. Below the text is a large empty rectangular box. At the bottom, there is a row of seven small images: a dog on a wooden deck, a black cat, a dog in a field, a dog in a field, a dog in a field, a dog in a field, and a dog in a field. Below the images is the text "What do these images have in common? Find out!"

Very general image classification task.

All changed with AlexNet and the 2012 ImageNet Challenge...

team name	team members	filename	flat cost	hie cost	description
NEC-UIUC	NEC: Yuanqing Lin, Fengjun Lv, Shenghuo Zhu, Ming Yang, Timothee Cour, Kai Yu UIUC: LiangLiang Cao, Zhen Li, Min-Hsuan Tsai, Xi Zhou, Thomas Huang Rutgers: Tong Zhang	flat_opt.txt	0.28191	2.1144	using sift and lbp feature with two non-linear coding representations and stochastic SVM, optimized for top-5 hit rate

2010 Results

Team name	Filename	Error (5 guesses)	Description
SuperVision	test-preds-141-146.2009-131- 137-145-146.2011-145f.	0.15315	Using extra training data from ImageNet Fall 2011 release
SuperVision	test-preds-131-137-145-135- 145f.txt	0.16422	Using only supplied training data
ISI	pred_FVs_wLACs_weighted.txt	0.26172	Weighted sum of scores from each classifier with SIFT+FV, LBP+FV, GIST+FV, and CSIFT+FV, respectively.

2012 Results

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto

kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto

ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto

hinton@cs.utoronto.ca

Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called "dropout" that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.



2019 Turing award winners

“For conceptual and engineering breakthroughs that have made deep neural networks a critical component of computing.”



Yann LeCun



Geoff Hinton



Yoshua Bengio

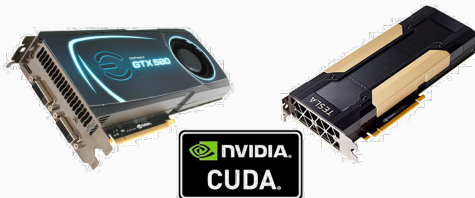
What were these breakthroughs? What made training large neural networks computationally feasible?

- Clever ideas in changing neural network architecture and training. E.g. ReLU non-linearities, dropout regularization, batch normalization, data augmentation.
- Wide-spread access to GPU computing power.

Graphics Processing Unit

Hardware innovation: Widely available, inexpensive GPUs allowing for cheap, highly parallel linear algebra operations.

- 2007: Nvidia released CUDA platform, which allows GPUs to be easily programmed for general purposed computation.



AlexNet architecture used 60 million parameters. Could not have been trained using CPUs alone (except maybe on a government super computer).

Two main algorithmic tools for training neural network models:

1. Stochastic gradient descent.
2. Backpropagation.

Training neural networks

Let $f(\boldsymbol{\theta}, \mathbf{x})$ be our neural network. A typical ℓ -layer feed forward model has the form:

$$g_{\ell}(\mathbf{W}_{\ell}(\dots \mathbf{W}_3 \cdot g_2(\mathbf{W}_2 \cdot g_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3 \dots) + \mathbf{b}_{\ell}).$$

$\mathbf{W}_i$ and $\mathbf{b}_i$ are the weight matrix and bias vector for layer i and g_i is the non-linearity (e.g. sigmoid). $\boldsymbol{\theta} = [\mathbf{W}_0, \mathbf{b}_0, \dots, \mathbf{W}_{\ell}, \mathbf{b}_{\ell}]$ is a vector of all entries in these matrices.

Goal: Given training data $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)$ minimize the loss

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^n L(y_i, f(\boldsymbol{\theta}, \mathbf{x}_i))$$

Example: We might use the binary cross-entropy loss for binary classification:

$$L(y_i, f(\boldsymbol{\theta}, \mathbf{x}_i)) = -y_i \log(f(\boldsymbol{\theta}, \mathbf{x}_i)) - (1 - y_i) \log(1 - f(\boldsymbol{\theta}, \mathbf{x}_i))$$

Gradient of the loss

Approach: minimize the loss by using gradient descent. Which requires us to compute the gradient of the loss function, $\nabla \mathcal{L}$. Note that this gradient has an entry for every value in $\mathbf{W}_0, \beta_0, \dots, \mathbf{W}_\ell, \beta_\ell$.

As usual, our loss function has finite sum structure, so:

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^n \nabla L(y_i, f(\boldsymbol{\theta}, \mathbf{x}_i))$$

So we can focus on computing:

$$\nabla_{\boldsymbol{\theta}} L(y_i, f(\boldsymbol{\theta}, \mathbf{x}_i))$$

for a single training example $(\mathbf{x}_i, y_i)$.

Chain rule review

For a scalar function $f(x)$, we write the derivative with respect to x as:

$$f'(x) = \frac{df}{dx} = \lim_{t \rightarrow 0} \frac{f(x+t) - f(x)}{t}$$

For a multivariate function $f(x, y, z)$ we write the partial derivative with respect to x as:

$$\frac{\partial f}{\partial x} = \lim_{t \rightarrow 0} \frac{f(x+t, y, z) - f(x, y, z)}{t}$$

Chain rule review

Let $y(x)$ be a function of x and let $f(y)$ be a function of y . The chain rule says that:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial y} \frac{\partial y}{\partial x}$$

$$\begin{aligned}\frac{\partial f}{\partial x} &= \lim_{t \rightarrow 0} \frac{f(y(x+t)) - f(y(x))}{t} \\ &= \lim_{t \rightarrow 0} \frac{f(y(x+t)) - f(y(x))}{y(x+t) - y(x)} \cdot \frac{y(x+t) - y(x)}{t} \\ &= \lim_{t \rightarrow 0} \frac{f(y(x) + c) - f(y(x))}{c} \cdot \frac{y(x+t) - y(x)}{t}\end{aligned}$$

where $c = y(x+t) - y(x)$. As long as $\lim_{t \rightarrow 0} y(x+t) - y(x) = 0$ then the first term equals $\frac{\partial f}{\partial y}$. The second term equals $\frac{\partial y}{\partial x}$.

Multivariable chain rule

Let $y(x), z(x), w(x)$ be functions of x and let $f(y, z, w)$ be a function of y, z, w .

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial y} \cdot \frac{\partial y}{\partial x} + \frac{\partial f}{\partial z} \cdot \frac{\partial z}{\partial x} + \frac{\partial f}{\partial w} \cdot \frac{\partial w}{\partial x}$$

Example: Let $y(x) = x^3$ and $z(x) = x^2$. Let $f(y, z) = y \cdot z$. Then:

$$\begin{aligned} \frac{\partial f}{\partial x} &= \left(\frac{\partial f}{\partial y} \cdot \frac{\partial y}{\partial x} \right) + \left(\frac{\partial f}{\partial z} \cdot \frac{\partial z}{\partial x} \right) \\ &= \end{aligned}$$

Gradient of the loss

Applying chain rule each partial derivative of the loss:

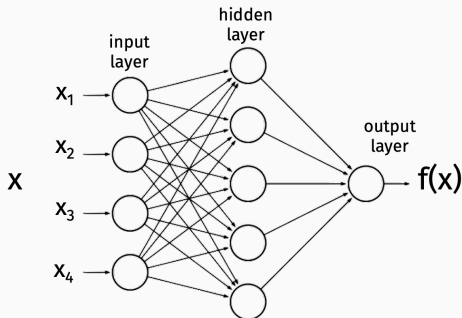
$$\nabla_{\theta} L(y, f(\theta, \mathbf{x})) = \frac{\partial L}{\partial f(\theta, \mathbf{x})} \cdot \nabla_{\theta} f(\theta, \mathbf{x})$$

Binary cross-entropy example:

$$L(y, f(\theta, \mathbf{x})) = -y \log(f(\theta, \mathbf{x})) - (1 - y) \log(1 - f(\theta, \mathbf{x}))$$

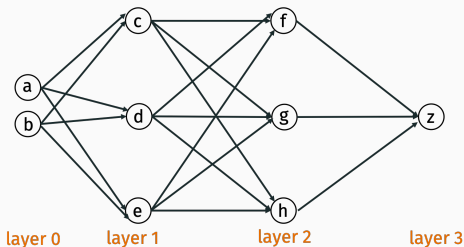
Gradient of the loss

We have reduced our goal to computing $\nabla_{\theta} f(\theta, \mathbf{x})$, where the gradient is with respect to the parameters θ .



Back-propagation is an efficient way to compute $\nabla_{\theta} f(\theta, \mathbf{x})$. It derives its name because we compute gradient from back to front: starting with the parameters closest to the output of the neural net.

Backprop example



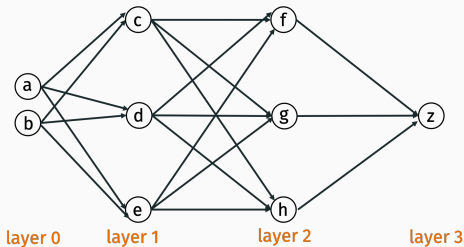
Notation for few slides:

- $a, b, \dots, z$ are the node names, and denote values at the nodes after applying non-linearity.
- $\bar{a}, \bar{b}, \dots, \bar{z}$ denote values before applying non-linearity.
- $W_{i,j}$ is the weight of edge from node i to node j .
- $s(\cdot) : \mathbb{R} \rightarrow \mathbb{R}$ is the non-linear activation function.
- β_j is the bias for node j .

Example: $h = s(\bar{h}) = s(c \cdot W_{c,h} + d \cdot W_{d,h} + e \cdot W_{e,h} + b_h)$

Backprop example

For any node j , let $\bar{j}$ denote the value obtained before applying the non-linearity s .



So if $h = s(c \cdot W_{c,h} + d \cdot W_{d,h} + e \cdot W_{e,h} + b_h)$ then we use $\bar{h}$ to denote:

$$\bar{h} = c \cdot W_{c,h} + d \cdot W_{d,h} + e \cdot W_{e,h} + b_h$$

Backprop example

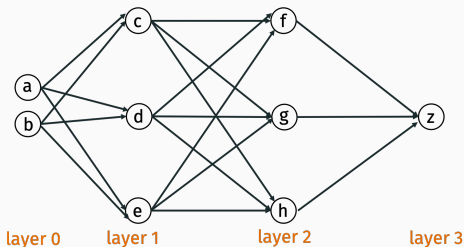
Goal: Compute the gradient $\nabla f(\theta, \mathbf{x})$, which contains the partial derivatives with respect to every parameter:

- $\partial z / \partial b_z$
- $\partial z / \partial W_{f,z}, \partial z / \partial W_{g,z}, \partial z / \partial W_{h,z}$
- $\partial z / \partial b_f, \partial z / \partial b_g, \partial z / \partial b_h$
- $\partial z / \partial W_{c,f}, \partial z / \partial W_{c,g}, \partial z / \partial W_{c,h}$
- $\partial z / \partial W_{d,f}, \partial z / \partial W_{d,g}, \partial z / \partial W_{d,h}$
- $\vdots$
- $\partial z / \partial W_{a,c}, \partial z / \partial W_{a,d}, \partial z / \partial W_{a,e}$

Two steps: Forward pass to compute function value. Backwards pass to compute gradients.

Backprop example

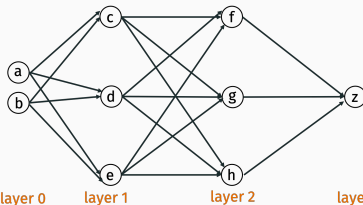
Step 1: Forward pass.



- Using current parameters, compute the output z by moving from left to right.
- Store all intermediate results:

$$\bar{c}, \bar{d}, \bar{e}, c, d, e, \bar{f}, \bar{g}, \bar{h}, f, g, h, \bar{z}, z.$$

Backprop example



Step 1: Forward pass.

$$\bar{c} = W_{a,c} \cdot a + W_{b,c} \cdot b + \beta_c \quad c = s(\bar{c})$$

$$\bar{d} = W_{a,d} \cdot a + W_{b,d} \cdot b + \beta_d \quad d = s(\bar{d})$$

$$\bar{e} = W_{a,e} \cdot a + W_{b,e} \cdot b + \beta_e \quad e = s(\bar{e})$$

$$\bar{f} = W_{c,f} \cdot c + W_{d,f} \cdot d + W_{e,f} \cdot e + \beta_f \quad f = s(\bar{f})$$

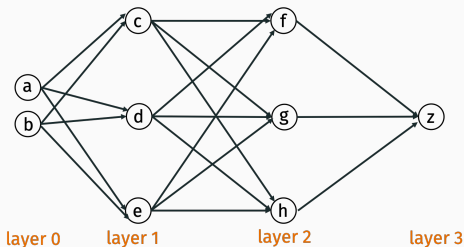
$$\vdots$$

$$\bar{z} = W_{f,z} \cdot f + W_{g,z} \cdot g + W_{h,z} \cdot h + \beta_z \quad z = s(\bar{z})$$

Question: What is runtime in terms of # of parameters P ?

Backprop example

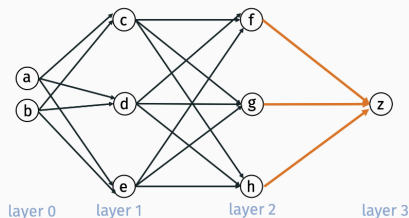
Step 2: Backward pass.



- Using **current parameters** and **computed node values**, compute the partial derivatives of all parameters by moving from right to left.

Backprop example

Step 2: Backward pass. Deepest layer.



$$\frac{\partial z}{\partial b_z} = \frac{\partial \bar{z}}{\partial b_z} \cdot \frac{\partial z}{\partial \bar{z}} = 1 \cdot s'(\bar{z})$$

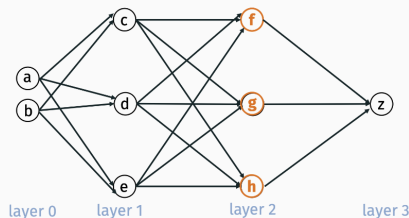
$$\frac{\partial z}{\partial W_{f,z}} = \frac{\partial \bar{z}}{\partial W_{f,z}} \cdot \frac{\partial z}{\partial \bar{z}} = f \cdot s'(\bar{z})$$

$$\frac{\partial z}{\partial W_{g,z}} = \frac{\partial \bar{z}}{\partial W_{g,z}} \cdot \frac{\partial z}{\partial \bar{z}} = g \cdot s'(\bar{z})$$

$$\frac{\partial z}{\partial W_{h,z}} = \frac{\partial \bar{z}}{\partial W_{h,z}} \cdot \frac{\partial z}{\partial \bar{z}} = h \cdot s'(\bar{z})$$

Backprop example

Step 2: Backward pass.



$$\frac{\partial z}{\partial f} = \frac{\partial \bar{z}}{\partial f} \cdot \frac{\partial z}{\partial \bar{z}} = W_{f,z} \cdot s'(\bar{z})$$

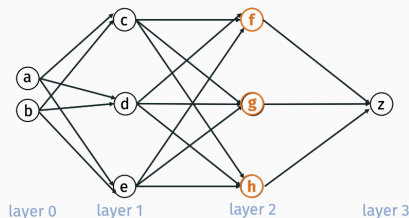
$$\frac{\partial z}{\partial g} = \frac{\partial \bar{z}}{\partial g} \cdot \frac{\partial z}{\partial \bar{z}} = W_{g,z} \cdot s'(\bar{z})$$

$$\frac{\partial z}{\partial h} = \frac{\partial \bar{z}}{\partial h} \cdot \frac{\partial z}{\partial \bar{z}} = W_{h,z} \cdot s'(\bar{z})$$

Compute partial derivatives with respect to nodes, even though these are not used in the gradient.

Backprop example

Step 2: Backward pass.

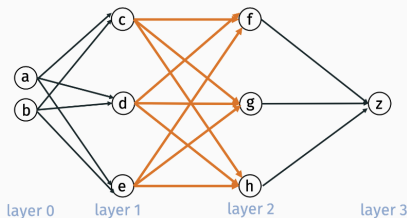


$$\begin{aligned}\frac{\partial z}{\partial \bar{f}} &= \frac{\partial z}{\partial f} \cdot \frac{\partial f}{\partial \bar{f}} = \frac{\partial z}{\partial f} \cdot s'(\bar{f}) \\ \frac{\partial z}{\partial \bar{g}} &= \frac{\partial z}{\partial g} \cdot \frac{\partial g}{\partial \bar{g}} = \frac{\partial z}{\partial g} \cdot s'(\bar{g}) \\ \frac{\partial z}{\partial \bar{h}} &= \frac{\partial z}{\partial h} \cdot \frac{\partial h}{\partial \bar{h}} = \frac{\partial z}{\partial h} \cdot s'(\bar{h})\end{aligned}$$

And for “pre-nonlinearity” nodes.

Backprop example

Step 2: Backward pass. Next layer.



$$\frac{\partial z}{\partial b_f} = \frac{\partial z}{\partial \bar{f}} \cdot \frac{\partial \bar{f}}{\partial b_f} = \frac{\partial z}{\partial \bar{f}} \cdot 1$$

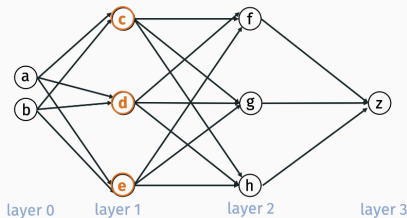
$$\frac{\partial z}{\partial W_{c,f}} = \frac{\partial z}{\partial \bar{f}} \cdot \frac{\partial \bar{f}}{\partial W_{c,f}} = \frac{\partial z}{\partial \bar{f}} \cdot c$$

$$\frac{\partial z}{\partial W_{d,f}} = \frac{\partial z}{\partial \bar{f}} \cdot \frac{\partial \bar{f}}{\partial W_{d,f}} = \frac{\partial z}{\partial \bar{f}} \cdot d$$

$$\frac{\partial z}{\partial W_{e,f}} = \frac{\partial z}{\partial \bar{f}} \cdot \frac{\partial \bar{f}}{\partial W_{e,f}} = \frac{\partial z}{\partial \bar{f}} \cdot e$$

Backprop example

Step 2: Backward pass. Next layer.



$$\begin{aligned}\frac{\partial z}{\partial c} &= \frac{\partial z}{\partial \bar{f}} \cdot \frac{\partial \bar{f}}{\partial c} + \frac{\partial z}{\partial \bar{g}} \cdot \frac{\partial \bar{g}}{\partial c} + \frac{\partial z}{\partial \bar{h}} \cdot \frac{\partial \bar{h}}{\partial c} \\ &= \frac{\partial z}{\partial \bar{f}} \cdot W_{c,f} + \frac{\partial z}{\partial \bar{g}} \cdot W_{c,g} + \frac{\partial z}{\partial \bar{h}} \cdot W_{c,h} \\ \frac{\partial z}{\partial d} &= \frac{\partial z}{\partial \bar{f}} \cdot W_{d,f} + \frac{\partial z}{\partial \bar{g}} \cdot W_{d,g} + \frac{\partial z}{\partial \bar{h}} \cdot W_{d,h} \\ \frac{\partial z}{\partial e} &= \frac{\partial z}{\partial \bar{f}} \cdot W_{e,f} + \frac{\partial z}{\partial \bar{g}} \cdot W_{e,g} + \frac{\partial z}{\partial \bar{h}} \cdot W_{e,h}\end{aligned}$$

Takeaways:

- Backpropagation can be used to compute derivatives for all weights and biases for any feedforward neural network.
- In the homework you will verify the final computation boils down to linear algebra operations (matrix multiplication and vector operations) which can be performed quickly on a GPU.